# Logistic Regression

## Fitted and Residuals

1

## In this lecture

- Fitted values $\hat{y}_i$
- Residuals $y_i - \hat{y}_i$
- Diagnostics



## Notation

To simplify the notation, let $L_i$ be the linear component and $p_i$ the conditional probability:

$$L_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \ldots + \beta_p X_{pi}$$

$L_i$: linear predictor (combine the predictors in a linear fashion)

and,

$$E\left[\, Y_i \mid \underline{X_{1i}, \ldots, X_{pi}} \,\right] = P\left[\, Y_i = 1 \mid \underline{X_{1i}, \ldots, X_{pi}} \,\right] = p_i$$

# Logistic Regression: equivalent forms

Scroll down to see full content

Recall that a logistic model can be written in different forms.

$$L_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \dots + \beta_p X_{pi}$$

$$p_i = \frac{e^{L_i}}{1 + e^{L_i}} \qquad \text{[probability]}$$

probability

most common form

$$\underbrace{\log\left(\frac{p_i}{1 - p_i}\right)}_{\textbf{the logit function}} = L_i \qquad \text{[log-odds]}$$

log(odds)

$$\underbrace{\left(\frac{p_i}{1 - p_i}\right)}_{\textbf{odds}} = e_i^L \qquad \text{[odds]}$$

equivalent

4

# Prediction

The estimated model can be used to predict an outcome

- the predicted outcome depends on the form of the model
  - log-odds (default, since this is a linear model)
  - probabilities
  - odds
- we can predict the outcome of:
  - observations in the same sample used to estimate the model, called **fitted** values $\hat{y}_i$
  - new observations, usually refered as predicted values $\hat{y}_i$



# Dataset and model

Let's use the `Titanic` dataset and an additive model with `sex` and `fare` as covariates to illustrate concepts.

```
model_titanic_logistic_additive <- glm(survived ~ sex + fare, data = titan,
                                       family = binomial)

tidy(model_titanic_logistic_additive) %>%
  mutate_if(is.numeric, round, 3) %>%
  knitr::kable()
```

| term | estimate | std.error | statistic | p.value |
|---|---|---|---|---|
| (Intercept) | 0.647 | 0.149 | 4.358 | 0 |
| sexmale | -2.423 | 0.171 | -14.208 | 0 |
| fare | 0.011 | 0.002 | 4.886 | 0 |

# Prediction

Scroll down to see full content

Suppose we want to use the predicted model to predict the odds of a male passenger who paid a fare of \$7.25.

The predicted *log-odds* are

$$\log\left(\frac{\hat{p}_s}{1-\hat{p}_s}\right) = \underbrace{0.647}_{\hat{\beta}_0} - \underbrace{2.423}_{\hat{\beta}_1} \times 1 + \underbrace{0.011}_{\hat{\beta}_2} \times 7.25 = -1$$

Next, by exponentiating both sides of the equation, we obtain

$$\frac{\hat{p}_s}{1-\hat{p}_s} = e^{-1.696} = 0.183$$

7

Finally, solving the above for $\hat{p}_s$, we obtain our predicted probability of surviving

$$\hat{p}_s = \frac{e^{-1.696}}{1+e^{-1.696}} = \frac{0.183}{1.183} = 0.155$$

# in R

## predicted log-odds (default)

```
predict(model_titanic_logistic_additive,
            tibble(sex = "male", fare = 7.25),
            type = "link") %>% round(3)
```

```
     1
-1.694
```

## predicted probabilities

```
predict(model_titanic_logistic_additive,
            tibble(sex = "male", fare = 7.25),
            type = "response") %>% round(3)
```

```
    1
0.155
```



# Fitted values

In R there are different ways of obtaining *fitted* values.

> It is important to know which forms are given by which function.

- the `augment()` function adds `.fitted` to the dataset: these are predicted *log-odds*
- the element `$fitted` or function `fitted()` of the estimated model object gives the predicted *probabilities*
- predicted *odds* can be computed from these columns

```r
model_titanic_logistic_additive %>%
            augment() %>%
            dplyr::select(survived,sex, fare,.fitted) %>%
            mutate(pred_prob1 = model_titanic_logistic_additive$fitted,
                   pred_prob2 = fitted(model_titanic_logistic_additive))
            head() %>%
            mutate_if(is.numeric, round, 3) %>%
            knitr::kable()
```

| survived | sex | fare | .fitted | pred_prob1 | pred_prob2 |
|---|---|---|---|---|---|
| 0 | male | 7.250 | -1.694 | 0.155 | 0.155 |
| 1 | female | 71.283 | 1.446 | 0.809 | 0.809 |
| 1 | female | 7.925 | 0.736 | 0.676 | 0.676 |
| 1 | female | 53.100 | 1.243 | 0.776 | 0.776 |
| 0 | male | 8.050 | -1.685 | 0.156 | 0.156 |
| 0 | male | 8.458 | -1.681 | 0.157 | 0.157 |

$y$ $x_1$ $x_2$

# Residuals

Scroll down to see full content

The **raw residuals** are, as usual, the differences between the observed and the fitted values.

$$
\begin{aligned}
r_i &= \text{obs}_i - \text{fit}_i \\
&= y_i - \hat{p}_i
\end{aligned}
$$

$\to 0, 1$

| $y$ | 0 | 1 |
|---|---|---|
| Prob. | $1-p$ | $p$ |

$E(y) = p = \frac{e^L}{1+e^L}$

$Var(y) = p(1-p)$

property of Bernoulli dist.

However, there are 2 problems with these residuals:

- the residuals are not properly scaled since the variance of the observations is not constant! for a binary response:

logistic model

$$Var(Y_i) = p_i(1 - p_i)$$

> technically, we should write the conditional variance and probability

$\to$ linear model:

$E(y) = L = \beta_0 + \beta_1 x$

$Var(y) = \sigma^2$ (separate parameter)

Thus, residuals of different observations are not comparable and should be adjusted!!

Because they also depend on variance of $y_i$



# Residuals for GLMs

Scroll down to see full content

Many variations of the raw residuals have been proposed for GLMs that take the variance of the observations into account.

- **Pearson residuals:** divides the raw residuals by the standard deviation of the response

$$r_i = \frac{y_i - \hat{p}_i}{\sqrt{\hat{p}_i(1 - \hat{p}_i)}}$$

more reliable than raw residuals $y_i - \hat{p}_i$

- **deviance residuals**

- **standardized residuals:** for both Pearson and deviance residual

We won't get to the details of these but it is important to know which ones you are calculating

```
model_titanic_logistic_additive %>%
              augment() %>%
              dplyr::select(survived,sex, fare,.fitted, .resid,.std.resid)
              mutate(pred_prob = model_titanic_logistic_additive$fitted,
                     resid_raw = residuals(model_titanic_logistic_additive,
                     raw_byhand = survived - pred_prob,
                     resid_deviance = residuals(model_titanic_logistic_addi
                     resid_pearson =residuals(model_titanic_logistic_additi
                     pearson_byhand = resid_raw/sqrt(pred_prob*(1-pred_prob
                     resid_standardized =rstandard(model_titanic_logistic_a
              head() %>%
              mutate_if(is.numeric, round, 3) %>%
              head(3) %>%
              knitr::kable()
```

| survived | sex | fare | .fitted | .resid | .std.resid | pred_prob | resid_raw | raw_by |
|---|---|---|---|---|---|---|---|---|
| 0 | male | 7.250 | -1.694 | -0.581 | -0.581 | 0.155 | -0.155 | -0.15 |
| 1 | female | 71.283 | 1.446 | 0.650 | 0.652 | 0.809 | 0.191 | 0.19 |
| 1 | female | 7.925 | 0.736 | 0.885 | 0.887 | 0.676 | 0.324 | 0.32 |

13

```
model_titanic_logistic_additive %>%
             aknitr::kable
             dplyr::select(survived,sex, fare,.fitted, .resid,.std.resid) %>%
             mutate(pred_prob = model_titanic_logistic_additive$fitted,
                    resid_raw = residuals(model_titanic_logistic_additive, type= "response"),
                    raw_byhand = survived - pred_prob,
                    resid_deviance = residuals(model_titanic_logistic_additive),
                    resid_pearson =residuals(model_titanic_logistic_additive,"pearson"),
                    pearson_byhand = resid_raw/sqrt(pred_prob*(1-pred_prob)),
                    resid_standardized =rstandard(model_titanic_logistic_additive)) %>%
             head() %>%
             mutate_if(is.numeric, round, 3) %>%
```

# Diagnostics

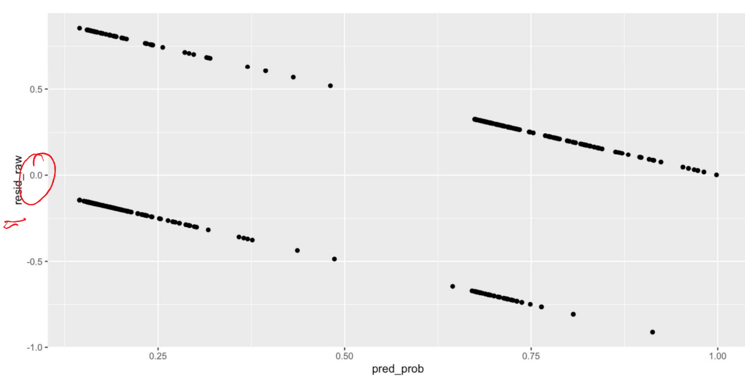
As we mentioned before, the raw residuals give only 2 values $-p_i$ or $1 - p_i$, which creates 2 straight lines in the residuals plots

Not much is gained using the Pearsons residuals, the problem stems from having a binary response

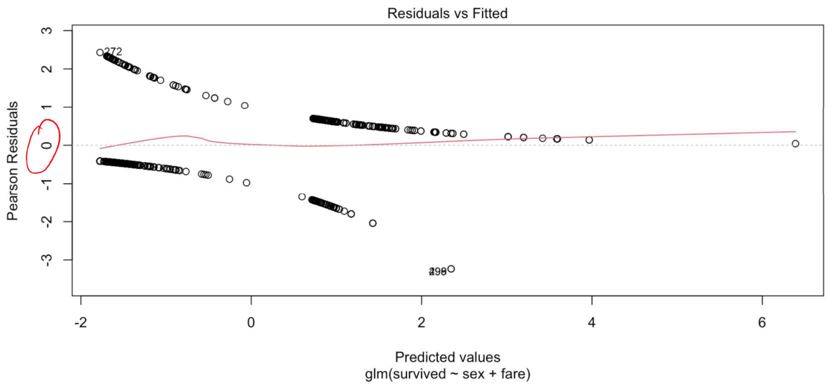
Although these are hard to interpret, we don't want to see clear patterns in a residuals plot.



```
model_titanic_logistic_additive %>%
          augment() %>%
          mutate(pred_prob = model_titanic_logistic_additive$fitted,
          ggplot(aes(x= pred_prob,y=resid_raw)) +
          geom_point()
```



```
1 plot(model_titanic_logistic_additive,1)
```



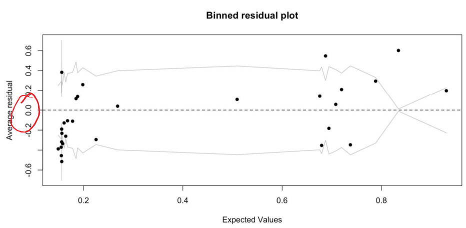

16

# Binned residuals plot

To address this problem, the binned plot has been proposed. Points are binned and the summary of each bin is plotted.

```
y_resid <- residuals(model_titanic_logistic_additive)
x_fit <- model_titanic_logistic_additive$fitted

residual_plot <- binnedplot(x_fit, y_resid)
```






# Overdispersion

the variance in the data is larger than theoretical variance $E(y_i)(1-E(y_i))$

Bernoulli dist

| Y | 0 | 1 |
|---|---|---|
| Prob | 1 | p |

model: $E(y) = L$

$Var(y) = E(y)(1-E(y))$ which may not be consistent with data

A more important problem to address in the case of logistic regression is the problem of **overdispersion**.

- As mentioned before, the logistic regression is built assuming that the response is *Bernoulli*, which means that the estimated variance of the response has to be $\hat{p}(1 - \hat{p})$.

- Unfortunately, in real applications, even in situations where the model seems to be estimating the mean well, the data's variability is not quite compatible with the model's assumed variance.

Solution: Assume $Var(y) = \lambda \cdot E(y)(1-E(y))$ where $\lambda$ is called a dispersion parameter.

We can estimate the value of $\lambda$ to see if it is close to 1 or not variance.

Solution: Assume $Var(y) = \lambda \cdot E(y)$ ... where $\lambda$ is called a dispersion par[ameter]. We can estimate the value of $\lambda$ to see if it is close to 1 or not

18

# Fixing the overdispersion

This misspecification in the variance affects the SE of the coefficients, not their estimates.

- A more flexible estimate of the model can be obtained using a method called *quasi-maximum likelihood*

- The estimated model also gives an estimate of the **overdispersion parameter**, as well as correct the standard error of our estimators.

_ If the model is correctly specified, the estimated **overdispersion parameter** is 1. Values above or below 1 indicate *over*- or *under*- dispersion

19

Scroll down to see full content

An easy implementation is obtained using the `family` argument to a `quasibinomial`.

```
summary(glm(
    formula = survived ~ sex + fare,
    data = titan,
    family = quasibinomial))
```

assume $Var(y) = \lambda E(y) \times (1 - E(y))$

```
Call:
glm(formula = survived ~ sex + fare, family = quasibinomial,
    data = titan)

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.647100   0.146953   4.403 1.20e-05 ***
sexmale     -2.422760   0.168736 -14.358  < 2e-16 ***
fare         0.011214   0.002271   4.937 9.47e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasibinomial family taken to be 0.9792377)
```

more reliable

check if $\hat{\lambda}$ is close to 1 or not

20

> Note that the estimated overdispersion parameter (called *Dispersion parameter* in R) is very close to 1 (0.98) indicating no signs of overdispersion.

$\Rightarrow$ Binomial assumption holds, so the results are valid.

Although there exists a formal test for overdispersion but we won't cover that in the course.